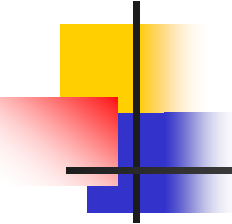


# 算法设计与分析

Analysis and Design of Algorithm

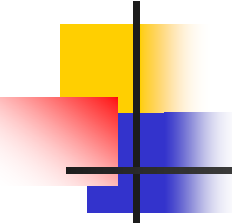
## Lesson 11



# 要点回顾

---

- 贪心算法几个实例：
  - 活动安排问题的正确性证明
  - 最优装载问题
  - 最优二元前缀码（Huffman树）
  - 图问题中的贪心算法
    - 最小生成树问题
      - Prim算法



# 求最小生成树

---

- **问题：**

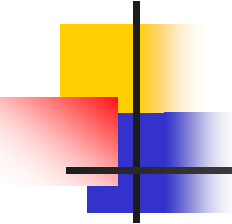
给定连通带权图 $G=(V,E,W)$ ,  $W(e) \in W$ 是边 $e$ 的权。求 $G$ 的一棵最小生成树。

- **贪心法：**

- Prim算法

- Kruskal算法

最小生成树有着重要应用！



# Prim算法

---

## ■ 设计思想

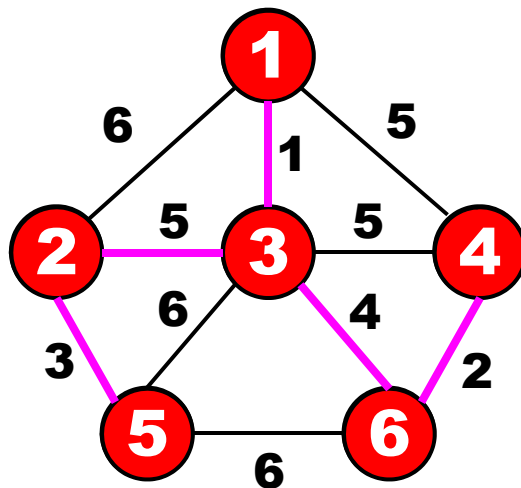
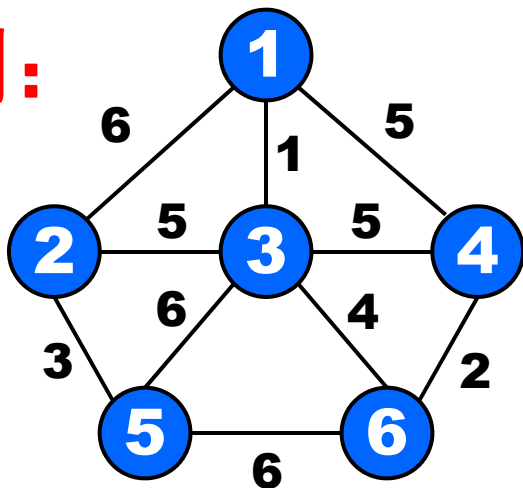
- 输入：图 $G=(V,E,W)$ ,  $V=\{1,2,\dots,n\}$
- 输出：最小生成树 $T$
- 步骤：
  - 初始 $S=\{1\}$ ;
  - 选择连接 $S$ 与 $V-S$ 集合的最短边 $e=\{i, j\}$ ，其中 $i \in S$ ,  $j \in V-S$ 。将 $e$ 加入树 $T$ ,  $j$ 加入 $S$ ;
  - 继续执行上述过程，直到 $S=V$ 为止。

# Prim算法伪码

## ■ 算法Prim(G,E,W)

1.  $S \leftarrow \{1\}$
2. **while**  $V-S \neq \emptyset$  **do**
3.     从 $V-S$ 中选择 $j$ 使得 $j$ 到 $S$ 中顶点的边权最小
4.      $S \leftarrow S \cup \{j\}$

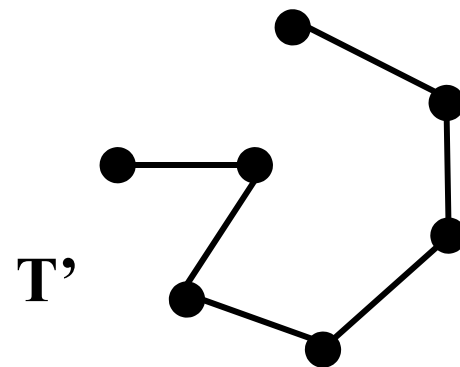
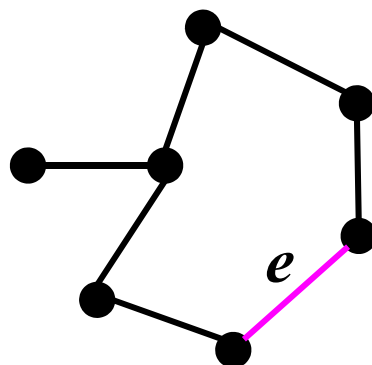
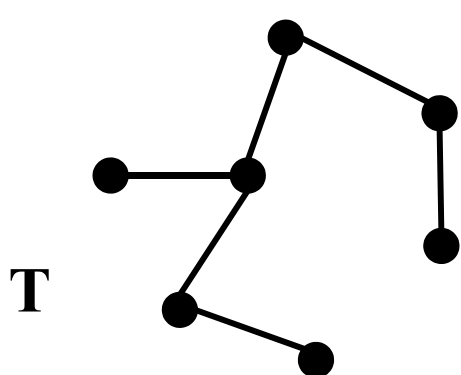
## ■ 实例:

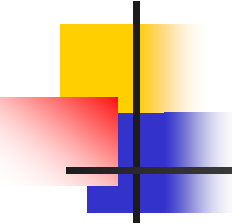


# 生成树的性质

**命题1:** 设 $G$ 是 $n$ 阶连通图, 那么

1.  $T$ 是 $G$ 的生成树当且仅当 $T$ 无圈且有 $n-1$ 条边;
2. 如果 $T$ 是 $G$ 的生成树,  $e \notin T$ , 那么 $T \cup \{e\}$ 含有一个圈 $C$ (回路)
3. 去掉圈 $C$ 的任意一条边, 就得到 $G$ 的另外一棵生成树 $T'$





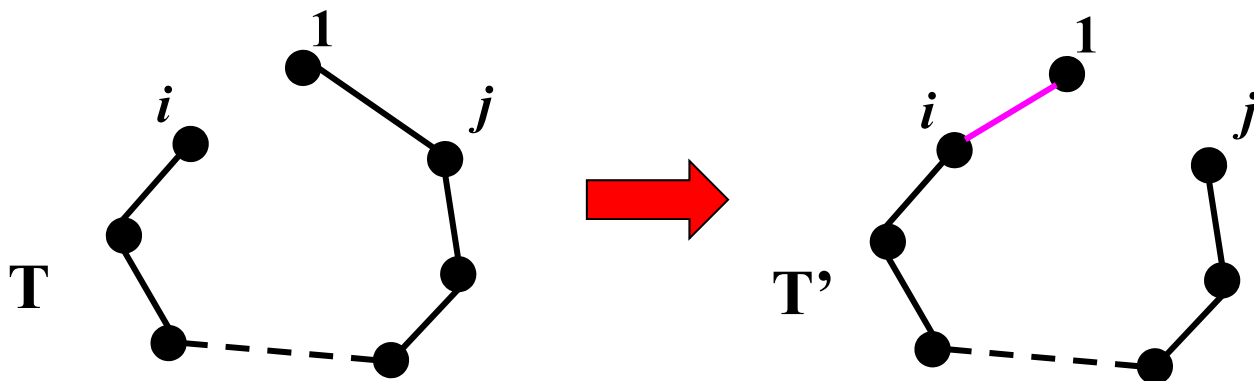
# 正确性证明：归纳法

- **命题：** 对于任意 $k < n$ ，存在一棵最小生成树包含算法前 $k$ 步选择的边。
- **归纳基础：**  $k=1$ ，存在一棵最小生成树 $T$ 包含边 $e=\{1,i\}$ ，其中 $\{1,i\}$ 是所有关联1的边中权最小的。
- **归纳步骤：** 假设算法前 $k$ 步选择的边构成一棵最小生成树的边，则算法前 $k+1$ 步选择的边也构成一棵最小生成树的边。

# 归纳基础

**证明：** 存在一棵最小生成树 $T$ 包含关联结点1的最小权的边 $e=\{1,i\}$ 。

**证** 设 $T$ 为一棵最小生成树，假设 $T$ 不包含 $\{1,i\}$ ，则 $T \cup \{1,i\}$ 含有一条回路，回路中关联1的另一条边 $\{1,j\}$ 。用 $\{1,i\}$ 替换 $\{1,j\}$ 得到树 $T'$ ，则 $T'$ 也是生成树，且 $W(T') \leq W(T)$ 。



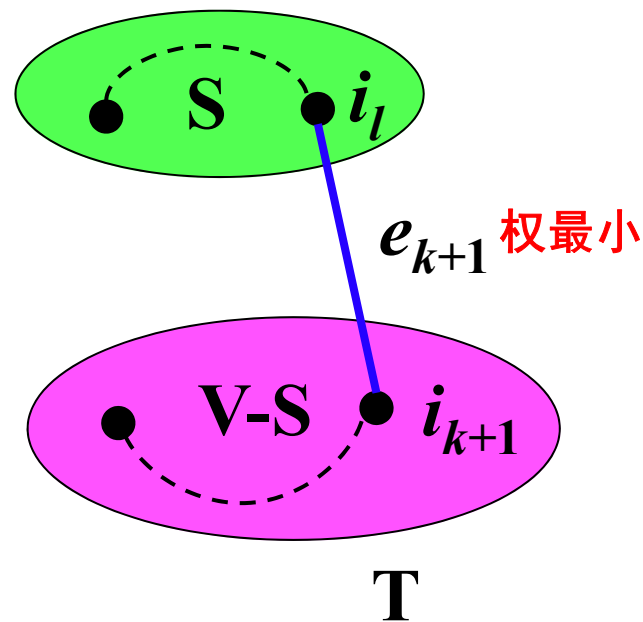


# 归纳步骤

假设算法进行了 $k$ 步，生成树的边为 $e_1, e_2, \dots, e_k$ ，这些边的端点构成集合 $S$ 。由**归纳假设**存在 $G$ 的一棵最小生成树 $T$ 包含这些边。

算法第 $k+1$ 步选择顶点 $i_{k+1}$ ，则 $i_{k+1}$ 到 $S$ 中顶点边权最小，设此边 $e_{k+1} = \{i_{k+1}, i_l\}$ 。

若 $e_{k+1} \in T$ ，算法 $k+1$ 步显然正确。



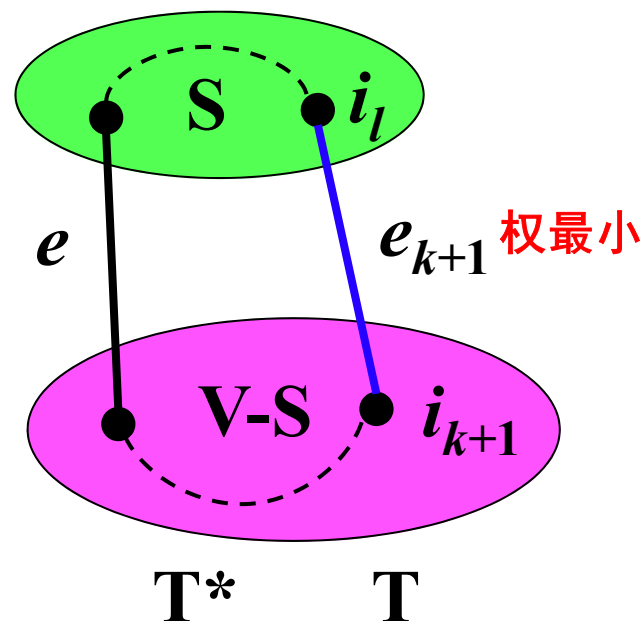
## 归纳步骤(cont.)

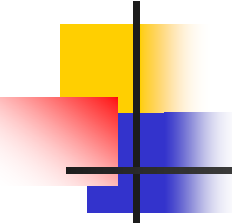
若  $e_{k+1} \notin T$ ，则将加到  $T$  形成一条回路。这条回路有另一条连接  $S$  与  $V-S$  中顶点的边  $e$

令  $T^* = (T - \{e\}) \cup \{e_{k+1}\}$ ，则  $T^*$  是  $G$  的一棵生成树，包含  $e_1, e_2, \dots, e_{k+1}$ ，且

$$W(T^*) \leq W(T)$$

算法到第  $k+1$  步仍得到最小生成树。

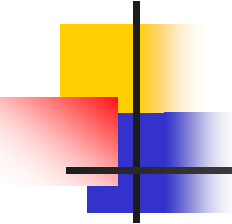




# 算法复杂度

---

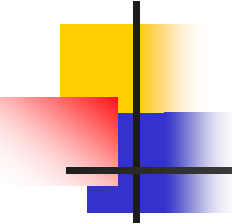
- 算法步骤执行 $O(n)$ 次
- 每次执行 $O(n)$ 时间：
  - 找连接S与V-S的最短边
- 算法时间： $T(n)=O(n^2)$



# Kruskal算法

## ■ 设计思想

- 输入：图 $G=(V,E,W)$ ,  $V=\{1,2,\dots,n\}$
- 输出：最小生成树 $T$
- 步骤：
  - 按照长度从小到大对边进行排序；
  - 依次考察当前最短边 $e$ ，如果 $e$ 与 $T$ 的边不构成回路，则把 $e$ 加入到树 $T$ ，否则跳过 $e$ 。直到选择了 $n-1$ 条边为止。

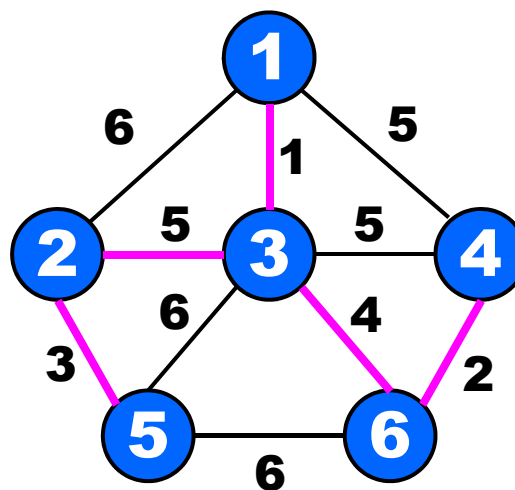
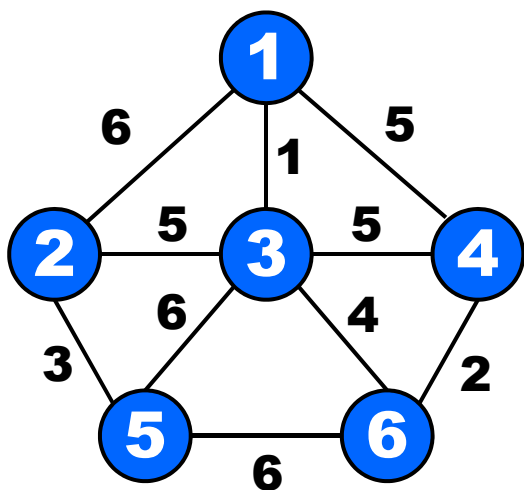


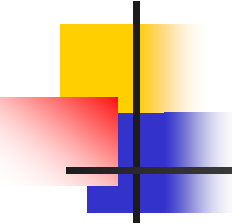
# Kruskal算法伪码

- 输入：图G //顶点数 $n$ ，边数 $m$
- 输出：最小生成树T
  1. 权从小到大排序E的边， $E=\{e_1, e_2, \dots, e_m\}$
  2.  $T \leftarrow \emptyset$
  3. repeat
  4.      $e \leftarrow E$ 中的最短边
  5.     if  $e$ 的两端点不在同一连通分支
  6.     then  $T \leftarrow T \cup \{e\}$
  7.      $E \leftarrow E - \{e\}$
  8. until T包含了 $n-1$ 条边

# Kruskal算法

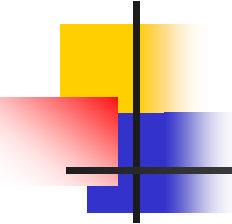
## ■ 实例





# 正确性证明思路

- **命题：**对于任意 $n$ ，算法对 $n$ 阶图找到一棵最小生成树。
- **证明思路**
  - **归纳基础** 证明： $n=2$ ，算法正确。（ $G$ 只有一条边，最小生成树就是 $G$ ）
  - **归纳步骤** 证明：假设算法对于 $n$ 阶图是正确的，其中 $n>1$ ，则对于任何 $n+1$ 阶图算法也得到一棵最小生成树。



# 思考

---

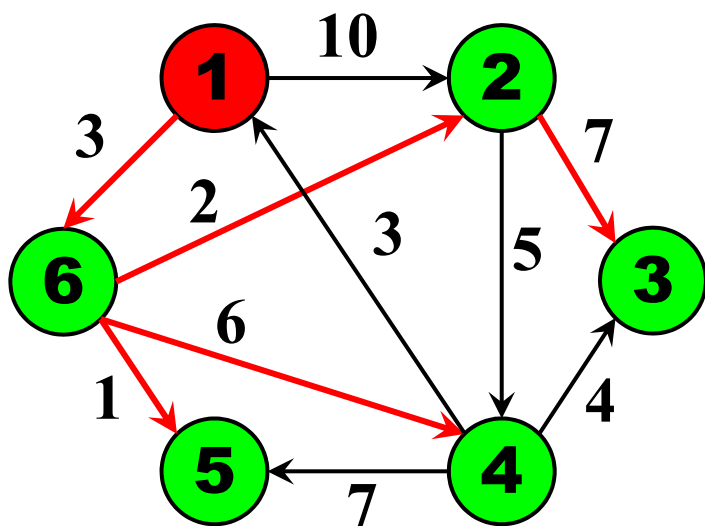
- 基于上述Prim和Kruskal算法思想
- 对于边数相对较多（即比较接近于完全图）的无向连通带权图，比较适合用哪种方法求解？
- 对边数较少的无向连通带权图有较高效率的又是哪一种算法？
- 请分析原因



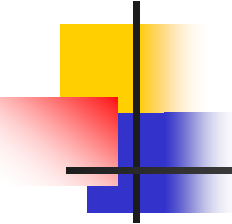
# 单源最短路径问题

给定带权有向图  $G=(V,E,W)$ ，每条边  $e=\langle i, j \rangle$  的权  $w(e)$  为非负实数，表示  $i$  到  $j$  的距离。源点  $s \in V$ 。

求：从  $s$  出发到达其他结点的最短路径。

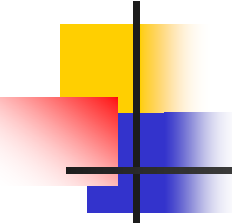


- 源点: 1
- $1 \rightarrow 6 \rightarrow 2$ :  $short[2]=5$
- $1 \rightarrow 6 \rightarrow 2 \rightarrow 3$ :  $short[3]=12$
- $1 \rightarrow 6 \rightarrow 4$ :  $short[4]=9$
- $1 \rightarrow 6 \rightarrow 5$ :  $short[5]=4$
- $1 \rightarrow 6$ :  $short[6]=3$



# Dijkstra算法有关概念

- $x \in S \Leftrightarrow x \in V$  且从  $s$  到  $x$  的最短路径已经找到
- 初始:  $S = \{s\}$ ,  $S = V$  时算法结束
- 从  $s$  到  $u$  相对于  $S$  的最短路径: 从  $s$  到  $u$  且仅经过  $S$  中顶点的最短路径
- $dist[u]$ : 从  $s$  到  $u$  相对  $S$  的最短路径的长度
- $short[u]$ : 从  $s$  到  $u$  的最短路径的长度
- $dist[u] \geq short[u]$



# 算法的设计思想

- **输入：** 有向图 $G=(V,E,W)$ ,  $V=\{1,2,\dots,n\}$ ,  $s=1$
- **输出：** 从 $s$ 到每个顶点的最短路径
- **步骤：**
  1. 初始 $S=\{1\}$ ;
  2. 对于 $i \in V-S$ , 计算1到 $i$ 的相对 $S$ 的最短路径, 长度记为 $dist[i]$ ;
  3. **选择** $V-S$ 中 $dist$ 值最小的 $j$ , 将 $j$ **加入**到 $S$ , **修改** $V-S$ 中的顶点的 $dist$ **值**;
  4. 继续上述过程, 直到 $S=V$ 为止。

# 算法伪码

## ■ 算法Dijkstra

1.  $S \leftarrow \{s\}$
2.  $dist[s] \leftarrow 0$
3. **for**  $i \in V - \{s\}$  **do**
4.      $dist[i] \leftarrow w(s,i)$  //初始化,  $s$ 到 $i$ 没边,  $w(s,i) = \infty$
5. **while**  $V - S \neq \emptyset$  **do**
6.     从 $V - S$ 取相对 $S$ 的最短路径顶点  $j$
7.      $S \leftarrow S \cup \{j\}$
8.     **for**  $i \in V - S$  **do**
9.         **if**  $dist[j] + w(i,j) < dist[i]$
10.             **then**  $dist[i] \leftarrow dist[j] + w(i,j)$

更新 $dist$ 值

# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1\}$

$dist[1]=0$

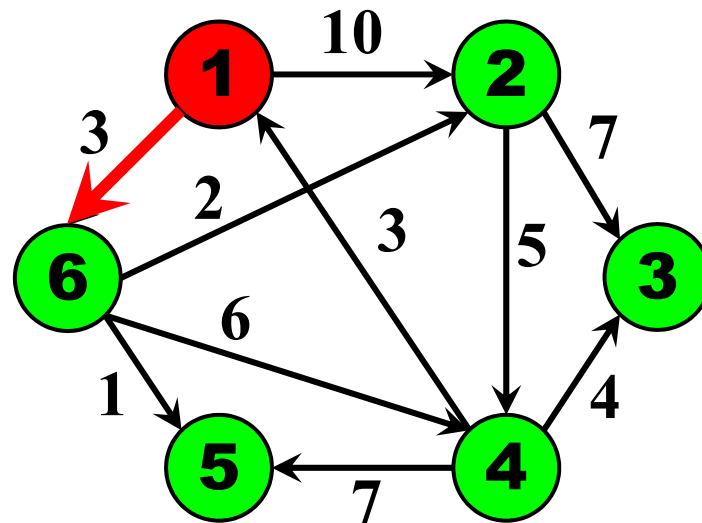
$dist[2]=10$

$dist[6]=3$

$dist[3]=\infty$

$dist[4]=\infty$

$dist[5]=\infty$



# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1,6\}$

$dist[1]=0$

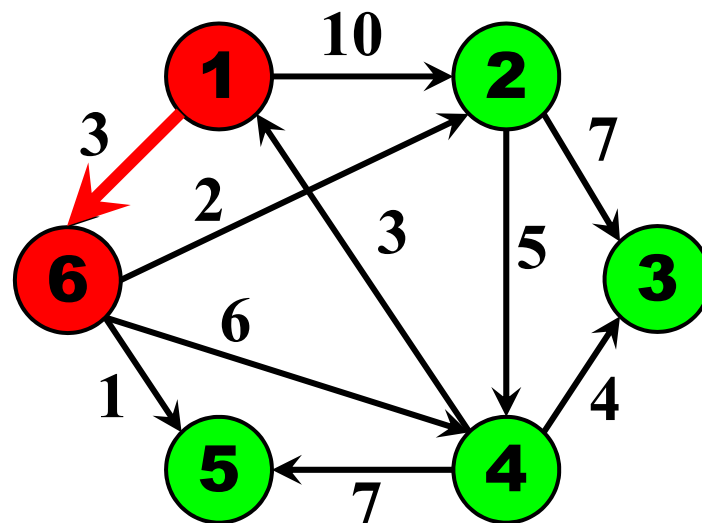
$dist[6]=3$

$dist[2]=5$

$dist[4]=9$

$dist[5]=4$

$dist[3]=\infty$



# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1,6,5\}$

$dist[1]=0$

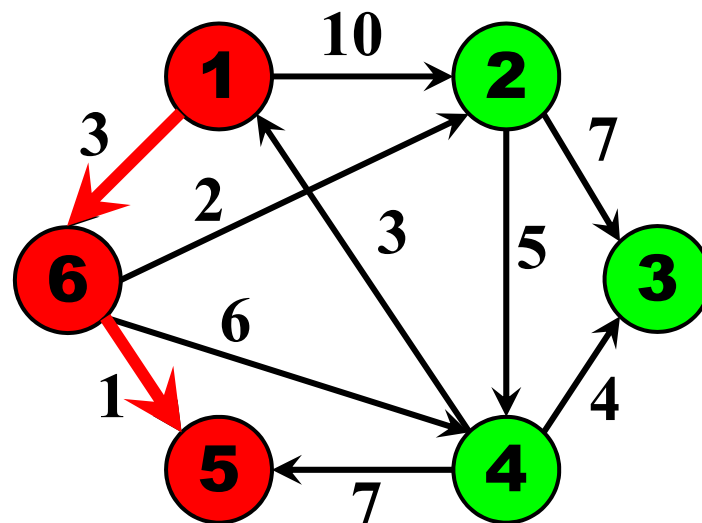
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=\infty$



# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1,6,5,2\}$

$dist[1]=0$

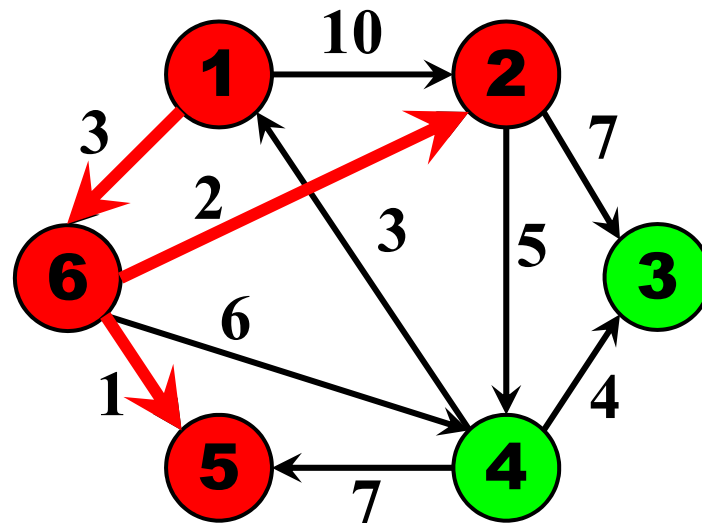
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=12$





# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1,6,5,2,4\}$

$dist[1]=0$

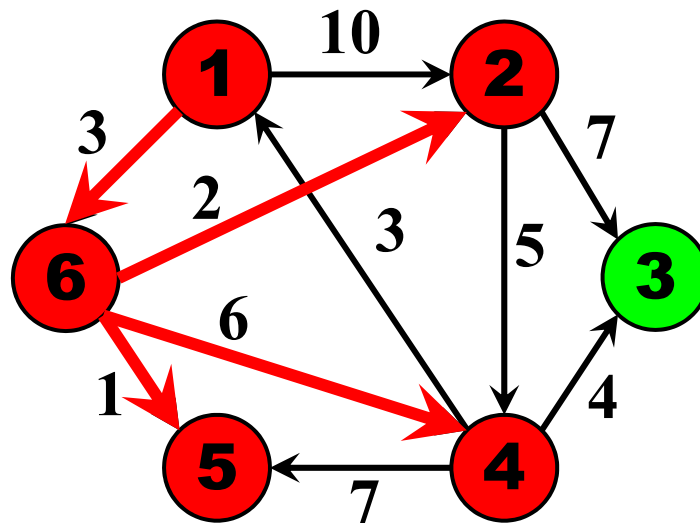
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=12$



# 运行实例

- 输入：  $G=(V,E,W)$ ,  $V=\{1,2,3,4,5,6\}$ , 源点1

$S=\{1,6,5,2,4,3\}$

$dist[1]=0$

$dist[6]=3$

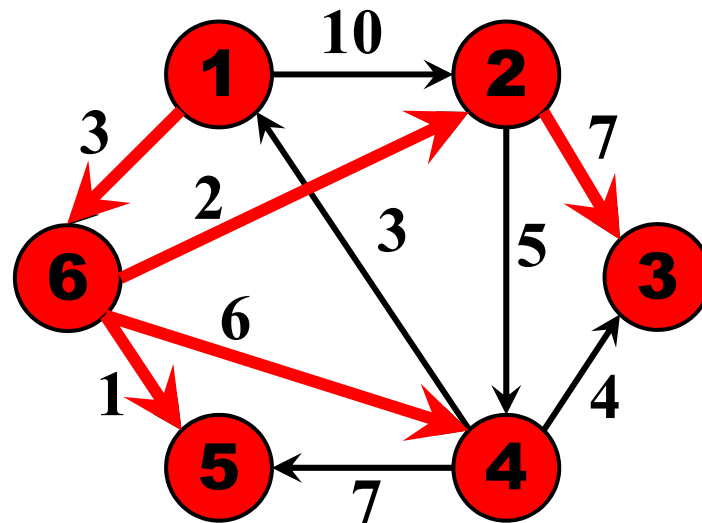
$dist[5]=4$

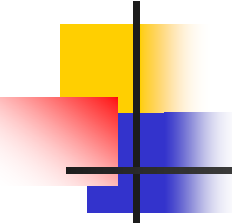
$dist[2]=5$

$dist[4]=9$

$dist[3]=12$

找到了问题的解！

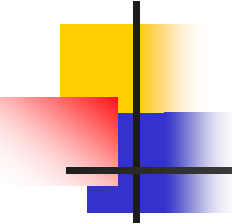




# 时间复杂度分析

- 时间复杂度：
  - 算法进行 $n-1$ 步
  - 每步挑选1个具有最小 $dist$ 函数值的结点进入到 $S$ ，需要 $O(m)$ 时间
  - $\rightarrow O(nm)$
- 选用基于堆实现的优先队列的数据结构，可以将算法时间复杂度降低到 $O(m\log n)$

# 贪心算法求解NP完全问题

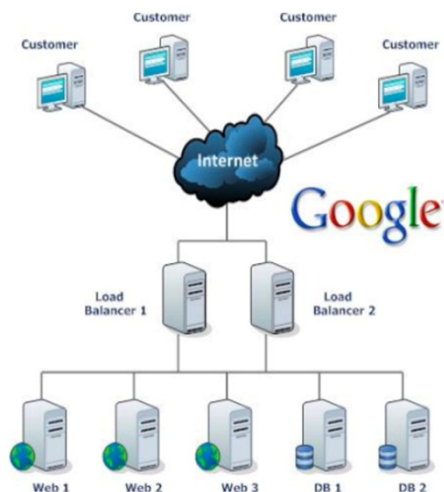


# 回顾：NP完全问题

- P问题的集合
  - 所有可以在多项式时间内求解的判定问题。
- NP问题的集合
  - 可用多项式时间的非确定性算法来进行判定或求解的问题。
- NP完全问题的集合
  - NP中某些问题不确定能否在多项式时间内求解。
  - 这些问题中，任何一个如果存在多项式时间的算法，那么所有NP问题都是多项式时间可解。
- **P=NP?**
  - 七个“千禧年数学难题”之一。

# 多机调度问题及其应用

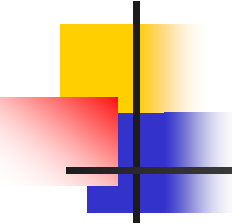
- **多机调度问题**要求给出一种作业调度方案，使所给的 $n$ 个作业，每个作业运行时间为 $t_i$ ，要求在尽可能短的时间内由 $m$ 台相同的机器加工处理完成。



计算机集群调度



工厂机床调度

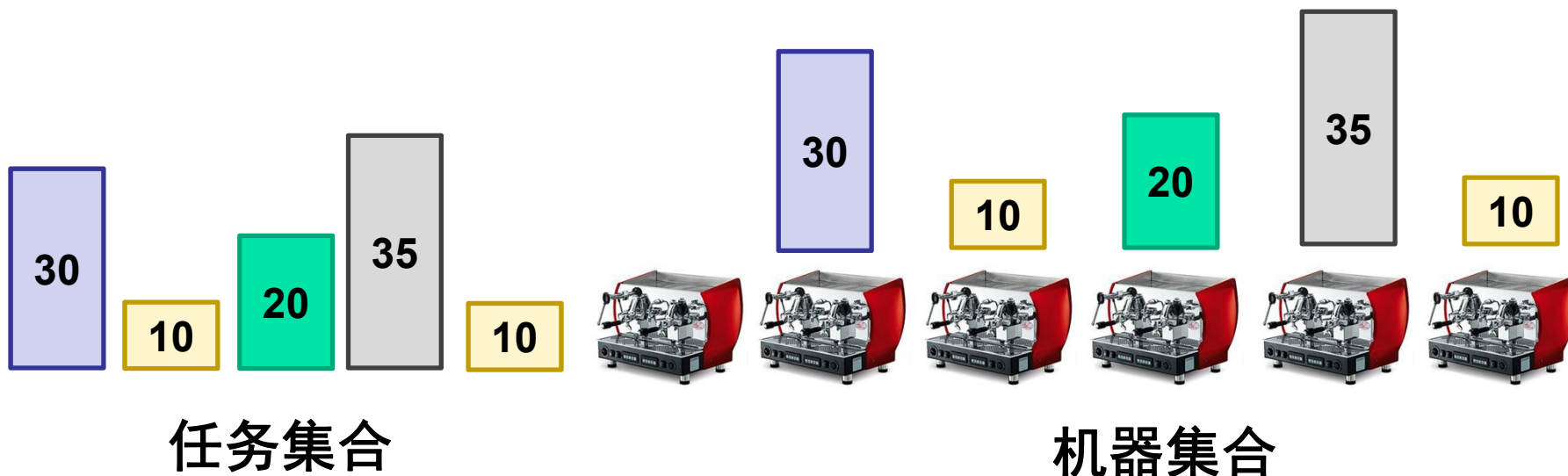


# 多机调度问题及其应用

- **多机调度问题**要求给出一种作业调度方案，使所给的 $n$ 个作业，每个作业运行时间为 $t_i$ ，要求在尽可能短的时间内由 $m$ 台相同的机器加工处理完成。
- 多机调度问题是**NP完全问题**，到目前为止还没有有效的解法。
- 对于这一类问题，用**贪心选择策略**有时可以设计出较好的**近似算法**。

# 多机调度问题—贪心策略

- 当机器数 $m \geq$ 任务数 $n$ 时
  - 每台机器放一个任务

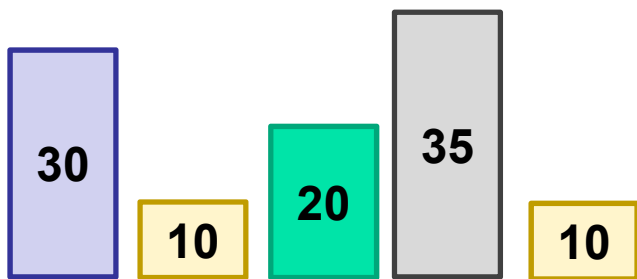




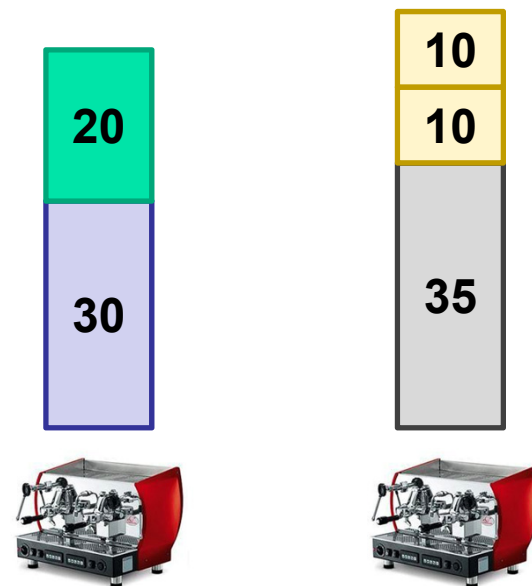
# 多机调度问题—贪心策略

- 当机器数 $m <$  任务数 $n$ 时
  - 优先放长任务
  - 优先选择负载最轻机器

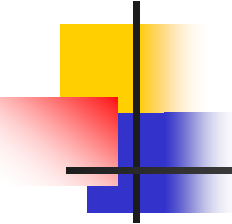
近似比等于多少？



任务集合



机器集合



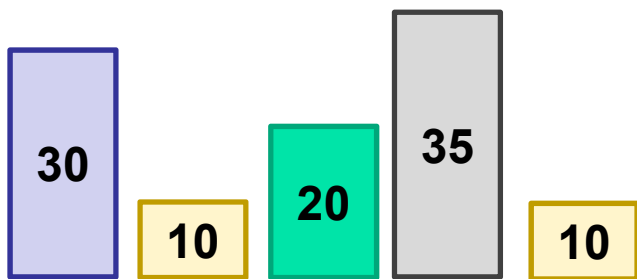
# 近似比(Approximation Ratio)

- 算法得到的近似解(ALG)与最优解(OPT)之间的比值，给出了近似解与最优解之间的相对误差的度量。是衡量近似算法质量的一个指标，特别是在解决NP难问题时，经常会使用近似算法来找到解的近似值。
- 最小化问题：  $r = \text{ALG}/\text{OPT}$
- 最大化问题：  $r = \text{OPT}/\text{ALG}$
- 近似比越接近1，表示近似算法找到的解与最优解越接近，算法的性能越好。

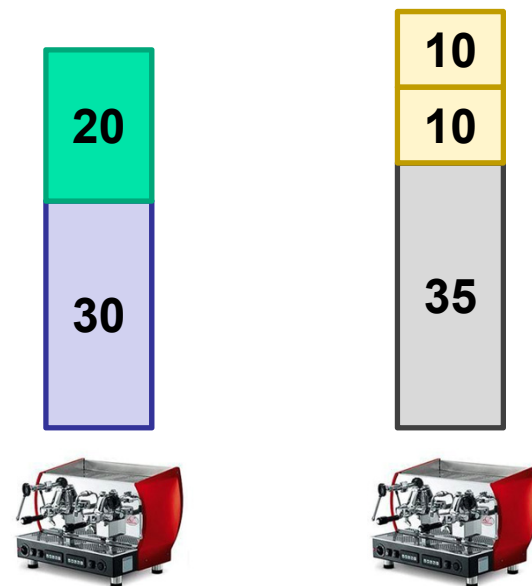
# 多机调度问题—贪心策略

- 当机器数 $m <$  任务数 $n$ 时
  - 优先放长任务
  - 优先选择负载最轻机器

近似比等于多少？



任务集合



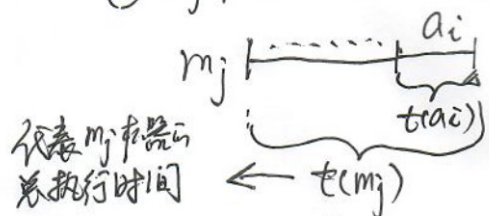
机器集合

# 贪心算法解多机调度问题的近似比推导

设有  $m$  台机器  $M = \{m_1, m_2, \dots, m_m\}$ ,  $n$  个任务  $A = \{a_1, a_2, \dots, a_n\}$ ,  $n > m$   
 现假设 最后完成任务的机器 为  $m_j$ , 且那个任务为  $a_i$ , 那么存在两种情况

①  $m_j$  机器上只有一个任务, 那么  $a_i$  肯定为  $a_1$ , 且是最优解 (记为 OPT)

②  $m_j$  机器上分面已不止一个任务, 那么  $a_i$  的下标  $i$  肯定  $\geq m+1 \Rightarrow \text{OPT} \geq 2t(a_i)$



代表任务  $a_i$  的执行时间  $\leftarrow t(a_i)$

另外, 理论上, 很容易分析得出两个结论 (1)  $\text{OPT} \geq \max_{a \in A} t(a)$ ; (2)  $\text{OPT} \geq \frac{1}{m} \sum_{a \in A} t(a)$

$$\text{那么就有 } t(m_j) - t(a_i) \leq \frac{1}{m} \left[ \sum_{a \in A} t(a) - t(a_i) \right]$$

即机器  $m_j$  在放入  $a_i$  前是  
最早空闲的机器

$$\Rightarrow t(m_j) \leq \frac{1}{m} \left[ \sum_{a \in A} t(a) \right] + \frac{t(a_i) - \frac{1}{m} t(a_i)}{1 - \frac{1}{m}}$$

$$\leq \text{OPT} \quad \quad \quad = (1 - \frac{1}{m}) t(a_i) \quad \text{而 } \text{OPT} \geq 2t(a_i) \Rightarrow t(a_i) \leq \frac{1}{2} \text{OPT}$$

$$\therefore t(m_j) \leq \text{OPT} + \frac{1}{2} (1 - \frac{1}{m}) \text{OPT} = (\frac{3}{2} - \frac{1}{2m}) \text{OPT}$$

$$\leq \frac{3}{2} \text{OPT}$$

即证出该贪心近似法的近似比为  $\frac{3}{2}$ .